A Statistical Model of Privacy-Budget Consumption in Repeated Aggregate SQL Workloads

CMP653 Project Final Presentation

Refik Can Öztaş

May 24, 2026

Hacettepe University
N25142279

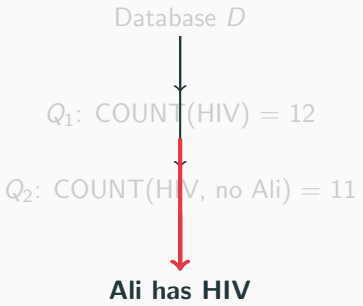
The Problem: Aggregate Queries Are Not Safe

Intuition (wrong): "Aggregate queries return totals, not individual rows – they must be safe."

Reality: Two overlapping COUNTs isolate one person.

```
-- 11 patients with HIV, age <> 43 SELECT COUNT(*)
FROM patients WHERE zip'06897' AND disease'HIV' AND
age <> 43;
==
```

⇒ The 43-year-old in 06897 has HIV.



Dinur & Nissim (2003) proved this generalizes: *O(n) noisy aggregates reconstruct the whole DB.*

Differential Privacy in 30 Seconds

Definition ($\varepsilon - DP$)
A randomized mechanism $\mathcal{M}$ is $\varepsilon - DPifforanytwoneighbouringdatabases\mathrm{D} \sim D'$ (differ by one row) and any output S:

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\varepsilon} \cdot \Pr[\mathcal{M}(D') \in S]$$

In practice (Laplace mechanism):

$$\tilde{f}(D) = f(D) + \eta, \quad \eta \sim \mathrm{Lap}(\Delta f/\varepsilon)$$

Budget composition: ε *accumulatesoverqueries*.

The Engineering Question
A finite budget $\varepsilon_{\text{total}}$ is consumed as the workload runs.

How does it degrade? Predicting this *before* deployment is what this project answers.

Where We Started: Milestone Version

What I built first:

- Python middleware: SQL parser, sensitivity analyzer, Laplace mechanism
- Budget ledger with **exact-repeat caching**: identical query $\Rightarrow$ reuse cached noisy answer ($\varepsilon = 0$)
- TPC-H benchmark, 4 workload families, 28 unit tests
- Headline number: **100$\times$ budget savings** on a repetitive workload

Milestone claim
"Workload-aware caching saves budget by exploiting DP's post-processing property."

Instructor's Verdict: Not Enough

Annotated PDF returned with:

- "What is the use of caching in DP?"
- "Algorithm 1 trivial!"
- "$1 - 1/k$?"
- "AVG – why?"
- "Caching is already working like that."

Summary review (the big one):

> *"Assumptions on exact-repeat reuse are very strong. Budget and temporality should be considered together. Leakage and savings models are very limited.* **Propose a statistical model.**"

The diagnosis

The milestone framed **caching as the contribution**.

But caching is a *direct consequence* of DP's post-processing property. Not novel.

What to do

Move from

mechanism ($\rightarrow$ trivial)

to

model ($\rightarrow$ contribution)

The Pivot: From Mechanism to Model

Old central claim: "caching saves budget" (Trivial, instructor flagged it.)

New central claim:

Research question (reframed)
Given a workload distribution and a privacy budget, can a **closed-form statistical model** predict the realized budget consumption and per-query utility *before any query is issued*?

Caching is now *the corner case* of the model, not the contribution.

Six new contributions:

1. Analytical model (**R2**)
2. Temporal extension (**R3**)
3. Middleware + 62 unit tests
4. Predictive allocator (**new mechanism**)
5. Leakage analysis (**R4**)
6. 4,155-trial experimental campaign (**R6**)

The Model in One Slide

Setup: workload of k queries drawn i.i.d. from a template distribution $\{p_i\}_{i=1}^{m}$.

Proposition 1 – Expected unique queries

$$\mathbb{E}[u_k] = \sum_{i=1}^{m} \left[1 - (1 - p_i)^k\right]$$

Coupon-collector / occupancy on a non-uniform distribution.

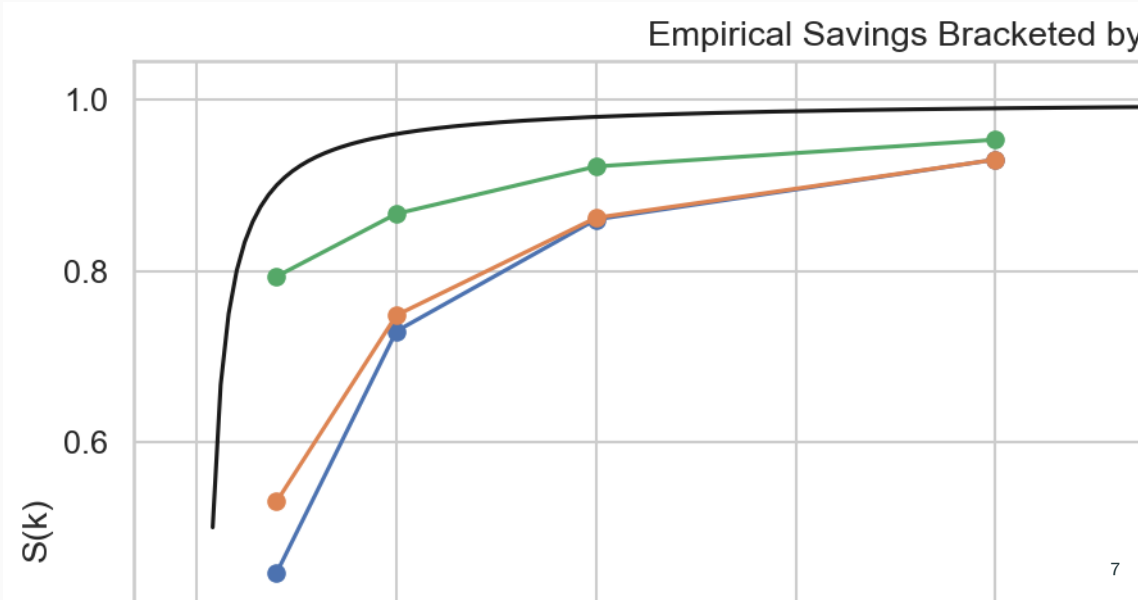
Proposition 2 – Workload-aware budget

$$\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \cdot \mathbb{E}[u_k] \qquad S(k) = 1 - \tfrac{1}{k} \sum_{i} \left[1 - (1 - p_i)^k\right]$$

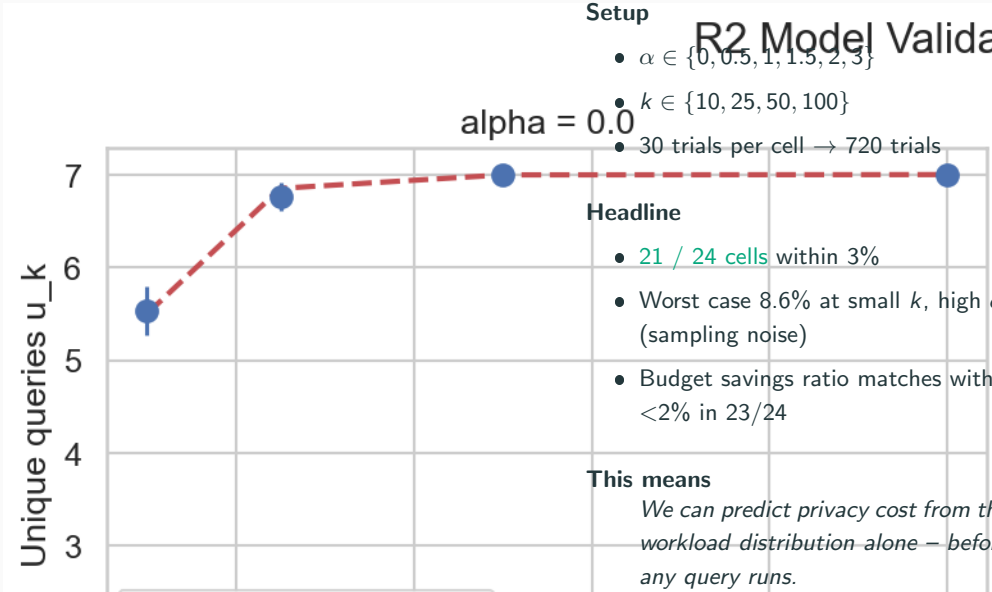
Two limits:

- $\alpha \to \infty$ (perfect repetition, $p_1 = 1$): $S(k) = 1 - 1/k$ (the milestone's toy formula)
- $\alpha \to 0$ (uniform, $m \to \infty$): $S(k) \to 0$ (naive composition recovered)

Empirical Validation: The Two Limits



Main Grid Validation: 21 / 24 Cells Within 3%



Setup

- $\alpha \in \{0, 0.5, 1, 1.5, 2, 3\}$
- $k \in \{10, 25, 50, 100\}$
- 30 trials per cell $\rightarrow$ 720 trials

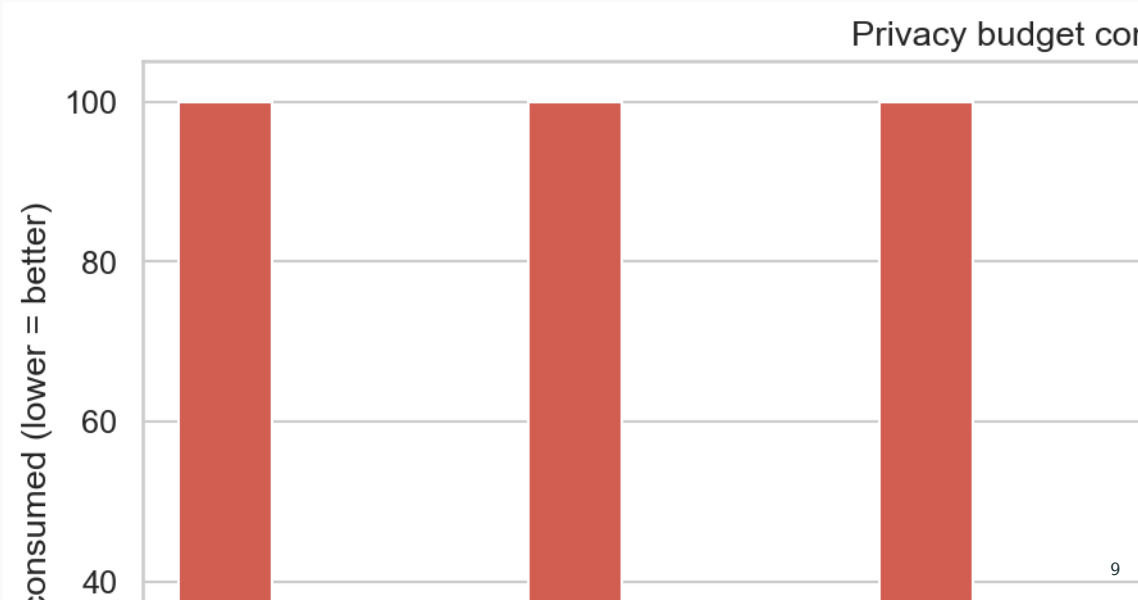
Headline

- 21 / 24 cells within 3%
- Worst case 8.6% at small k, high α (sampling noise)
- Budget savings ratio matches within $<$2% in 23/24

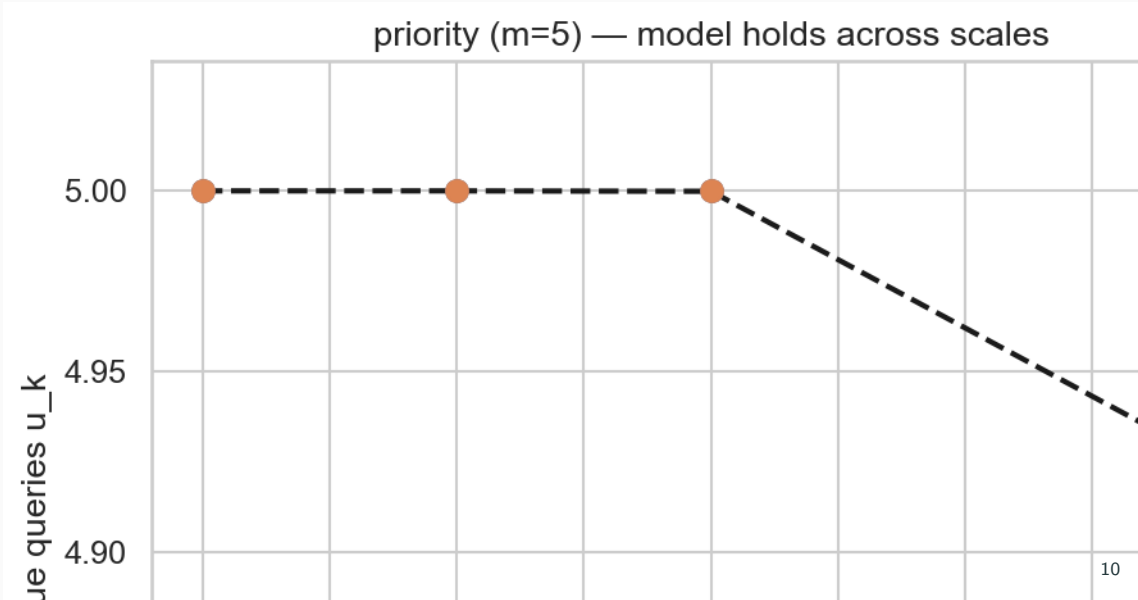
This means

We can predict privacy cost from the workload distribution alone – before any query runs.

Full Benchmark: 6 Workloads × 3 Modes



The Model Is Scale-Independent (SF=1 vs SF=10)



Mixed Heterogeneous Workload: Exact Match

W1 + W4 hybrid: fraction f repetitive, $1 - f$ unique.
By construction, true $u_k = 1 + (1 - f) \cdot k$.

f	true u_k	WORKLOAD ε used	predicted $\varepsilon_q \cdot u_k$
0.1	91	45.5	45.5
0.3	71	35.5	35.5
0.5	51	25.5	25.5
0.7	31	15.5	15.5
0.9	11	5.5	5.5

Empirical matches prediction to the cent in every cell.

The closed form $\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \cdot u_k$ holds without requiring the workload to be Zipf-parametric.

Putting the Model to Work: Predictive Allocator

The new mechanism: Use the model's $\mathbb{E}[u_k]$ estimate at runtime to choose ε_q.

1. Observe template frequencies online
2. After warmup, estimate $\hat{U} = \mathbb{E}[u_k \mid \hat{p}]$ from Proposition 1
3. Allocate $\varepsilon_q^{\text{pred}} = B/\hat{U}$

Novel because: PINQ, PrivateSQL, Chorus, DOP-SQL all fix ε_q offline.
This is the first DP-SQL mechanism that adapts ε_q at runtime from a learned model.

B	Mode	MAE	vs naive
1	NAIVE	101.1	—
	PREDICTIVE	**68.7**	−32%
10	NAIVE	10.1	—
	PREDICTIVE	**8.3**	−18%
50	NAIVE	2.0	—

AI / NLP Bonus: Semantic Cache via Tree Kernel + AST Embedding

Idea: two queries with the same AST (modulo literals) might be safe to share a cache entry.

Two similarity signals:

- **Tree Kernel** (Collins–Duffy): count shared subtrees – deterministic, no training
- **AST Embedding** (sentence-transformers / CodeBERT): dense vector, cosine similarity

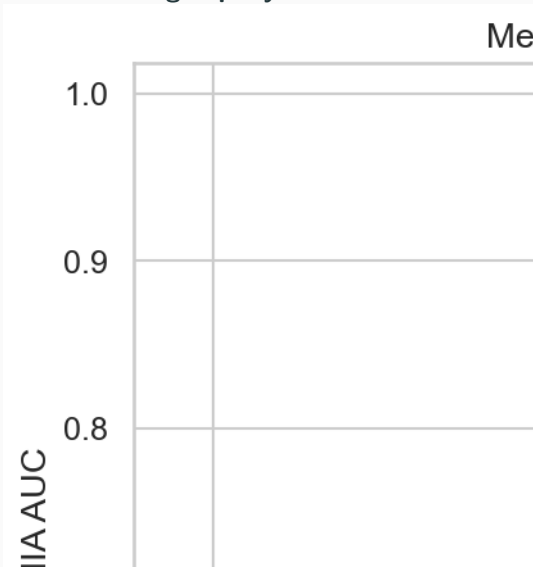
Honest negative result
The semantic L2 cache fails in **both directions**:

- **False positives:** reuses cached noisy answer on queries with different true answers $\Rightarrow$ wrong by thousands
- **False negatives:** misses textbook equivalences (commutative AND, integer comparators)

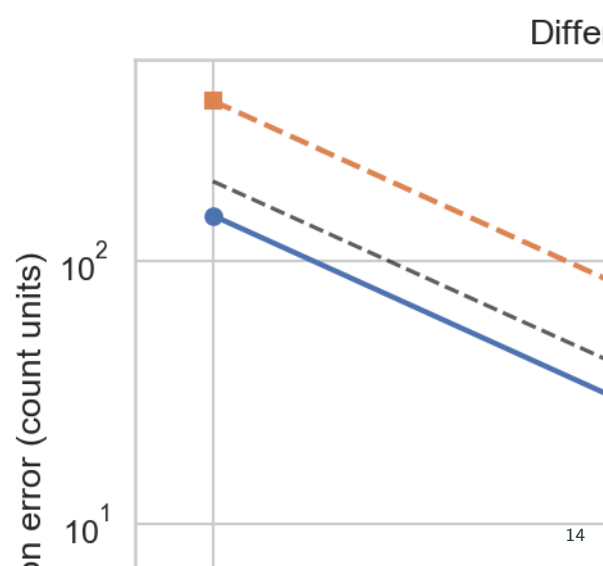
Conclusion: Semantic similarity $\neq$ formal equivalence.
"AI-powered DP cache" needs a *symbolic equivalence prover* on top.

Privacy Leakage: Quantified

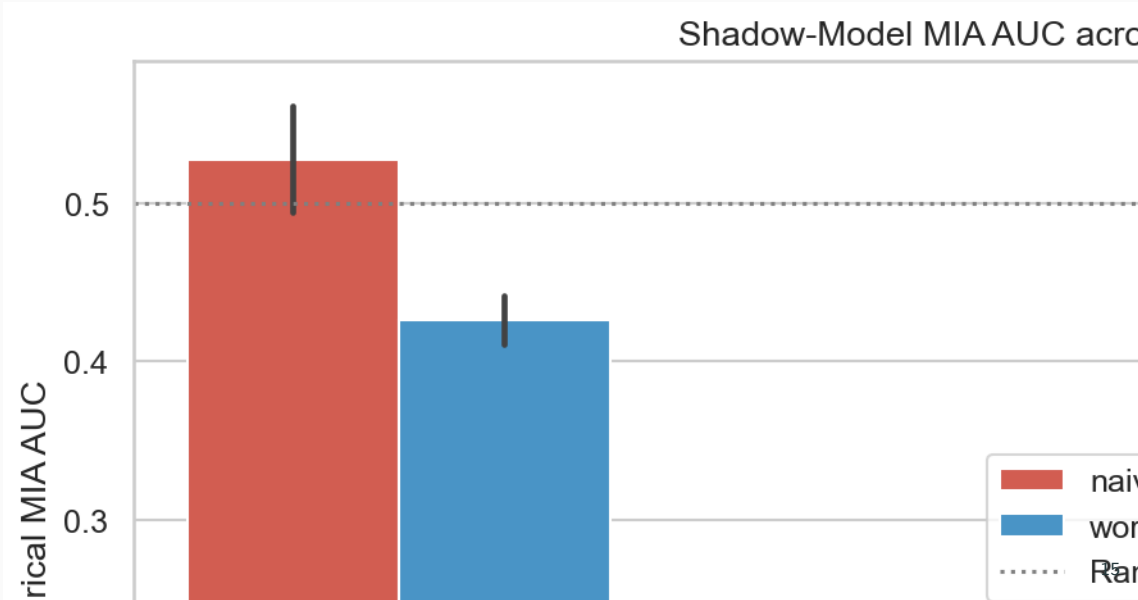
Single-query MIA AUC



Reconstruction Error



Workload-Level Shadow MIA: A Loose Bound



What We Found – Four Things Worth Stating

1. **Workload-aware budget is a structural property, not a cache trick.**

   $\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \cdot \mathbb{E}[u_k]$ predicts within 3% on the main grid, *exact* on the mixed W1+W4 workload, and the same at SF=1 and SF=10.

2. **The toy $1 - 1/k$ rule is the $\alpha \to \infty$ corner case.**

   Not a standalone claim – the model recovers it and the entire Zipf interior between it and naive composition.

3. **Cumulative-ε overestimates real attack success.**

   Shadow MIA on W1–W4 stays at AUC 0.15–0.58 vs. a 1.0 cumulative bound. Budget tends to be *over*-provisioned in practice.

4. **Semantic similarity is not equivalence.**

   Tree-kernel + embedding admits dangerous false positives and misses textbook equivalences. "AI-powered DP" alone is not DP-safe.

Honest Limitations & Future Work

What this project is not:

- *Not* a head-to-head comparison against real DP-SQL systems (PINQ, PrivateSQL, Chorus, DOP-SQL re-implemented from scratch is multi-month work)
- *Not* a real-workload trace – Zipf parametrization is a controlled sweep
- *Not* a new privacy mechanism in the cryptographic sense

Concrete next steps (paper §11):

- Cache re-release on budget surplus (closes predictive's main gap)
- Rényi DP / zCDP composition (tighter bound)
- Burstiness via renewal processes
- Real workload traces (Snowflake / Redshift anonymized)
- Joint budget–leakage bound that beats cumulative-ε
- Equivalence-checked semantic cache (similarity + symbolic prover)
- Multi-table / JOIN via residual sensitivity or R2T

Thank You

Refik Can Öztaş N25142279

Repo: `github.com/canoztas/cmp653-project`

4,155 core trials • 62 unit tests • 13-section paper

Questions?